
CS 301

High-Performance Computing

Assignment 05

Parallel Insertion and

Deletion in Mover

Khushi Pandya (202301406)
Aalok Thakkar (202301429)

April 1, 2026

Contents

1	Introduction	3
2	Theoretical Background	3
2.1	Particle Motion	3
2.2	Deletion Condition	3
2.3	Insertion	3
3	Parallelization Theory	3
3.1	Speedup	3
3.2	Amdahl's Law	3
3.3	Memory vs Compute Bound	3
4	Experiment 01: Scaling	3
4.1	Expected Behavior	3
5	Experiment 02: Speedup	6
5.1	Expected Behavior	6
5.2	Deferred	7
5.3	Immediate	10
6	Memory and Computation Analysis	12
6.1	Memory Requirement	12
6.2	Estimated FLOPs	12
6.3	Analysis Table	13
6.4	Discussion	13
7	Per-Particle Performance	14
8	Scalability Analysis	14
9	Performance Comparison	15
10	Flow Diagram	16
11	Bonus Questions	16
11.1	Data Structures	16
11.2	Optimizations	16
12	Conclusion	17

1 Introduction

This assignment extends the particle interpolation and mover algorithm by introducing dynamic particle insertion and deletion. Particles leaving the domain are deleted and replaced, maintaining a constant particle count.

2 Theoretical Background

2.1 Particle Motion

$$x \leftarrow x + r_x, \quad y \leftarrow y + r_y$$

$$r_x \in [-\Delta x, \Delta x], \quad r_y \in [-\Delta y, \Delta y]$$

2.2 Deletion Condition

$$x < 0 \text{ or } x > 1 \text{ or } y < 0 \text{ or } y > 1$$

2.3 Insertion

$$x, y \sim U(0, 1)$$

3 Parallelization Theory

3.1 Speedup

$$S = \frac{T_2}{T_n}$$

3.2 Amdahl's Law

$$S_{max} = \frac{1}{(1 - P) + \frac{P}{N}}$$

3.3 Memory vs Compute Bound

The algorithm becomes memory-bound at large particle sizes due to heavy memory access.

4 Experiment 01: Scaling

4.1 Expected Behavior

Execution time scales linearly:

$$O(N)$$

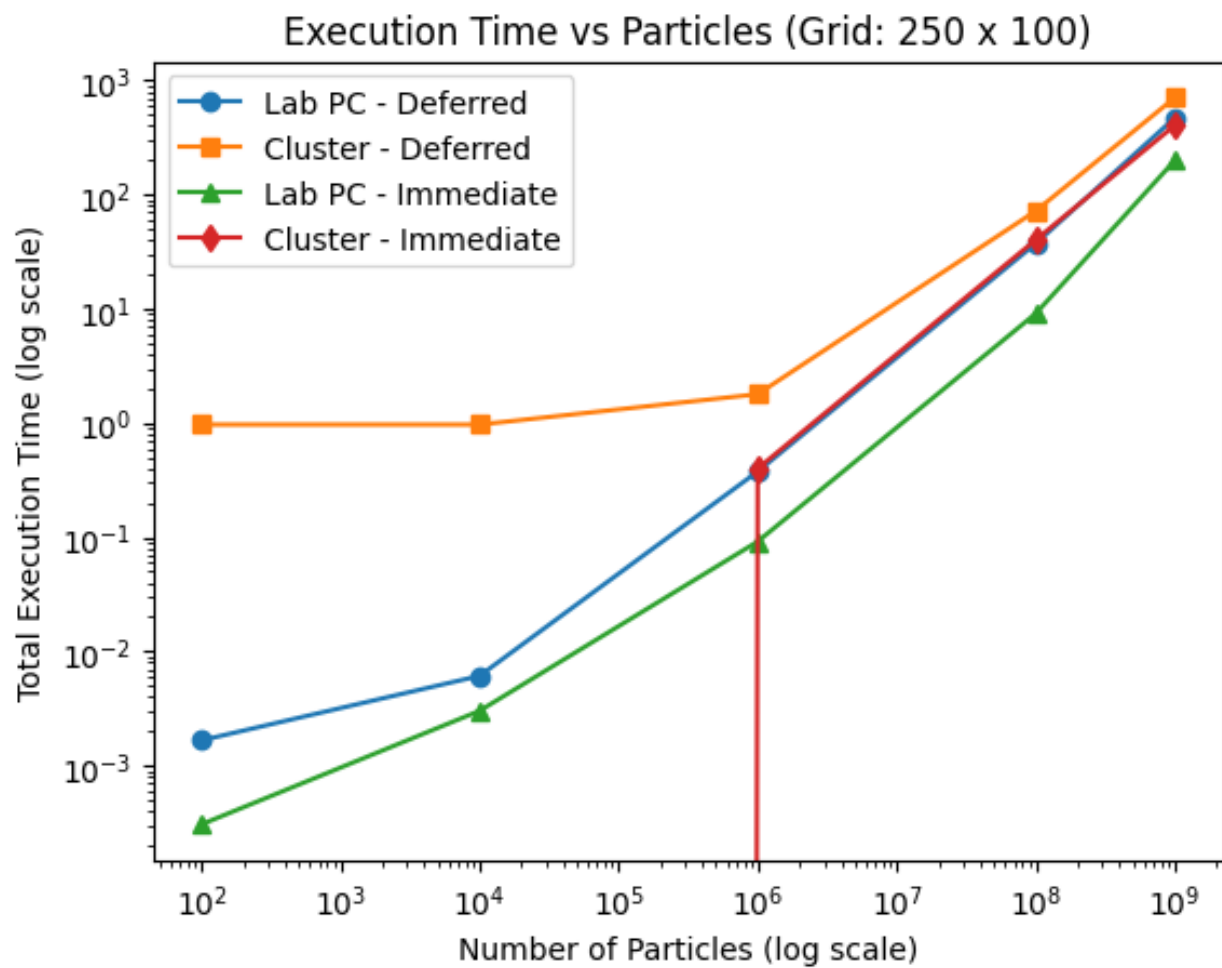


Figure 1: Grid 250x100

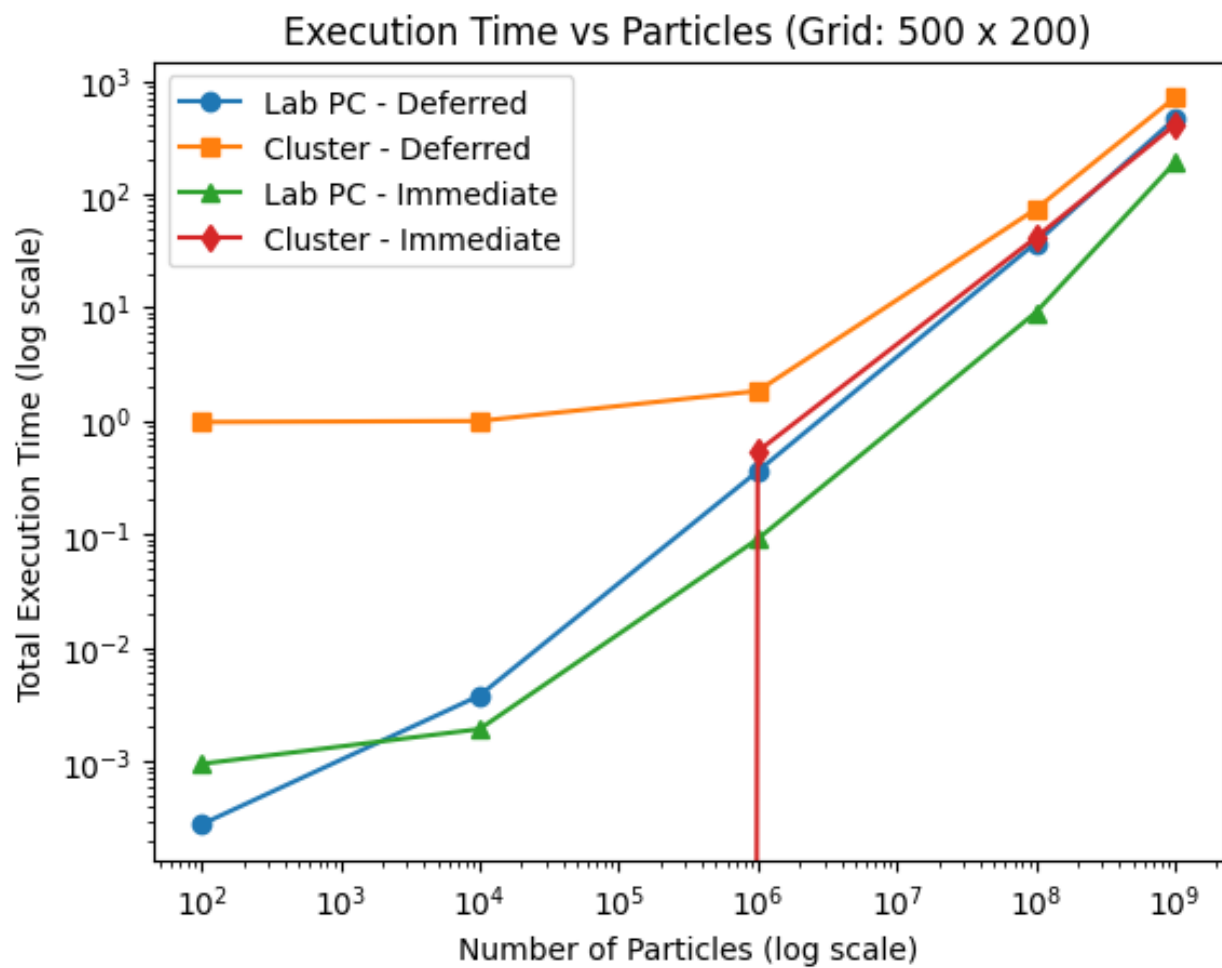


Figure 2: Grid 500x200

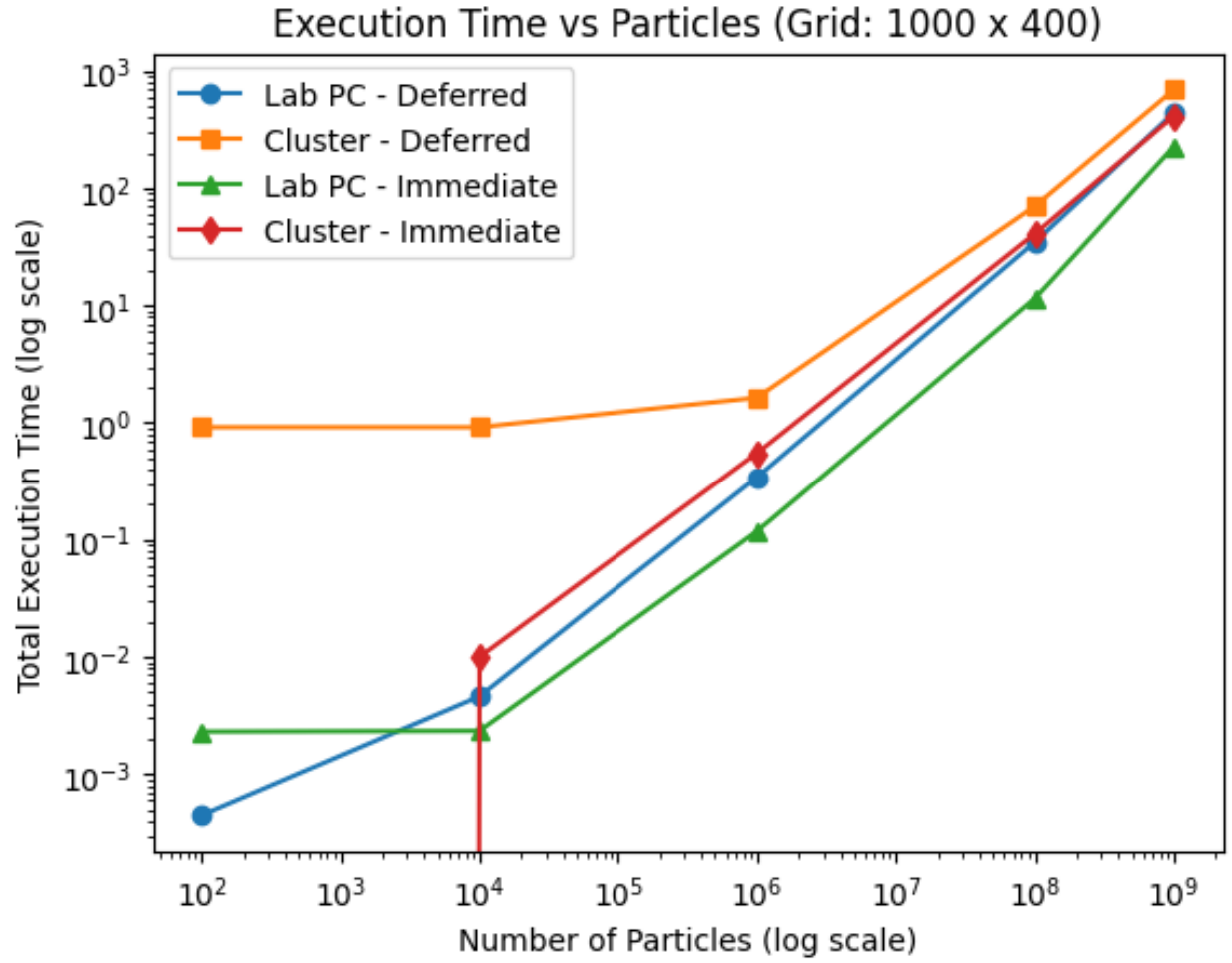


Figure 3: Grid 1000x400

5 Experiment 02: Speedup

5.1 Expected Behavior

Ideally:

$$S \propto N$$

5.2 Deferred

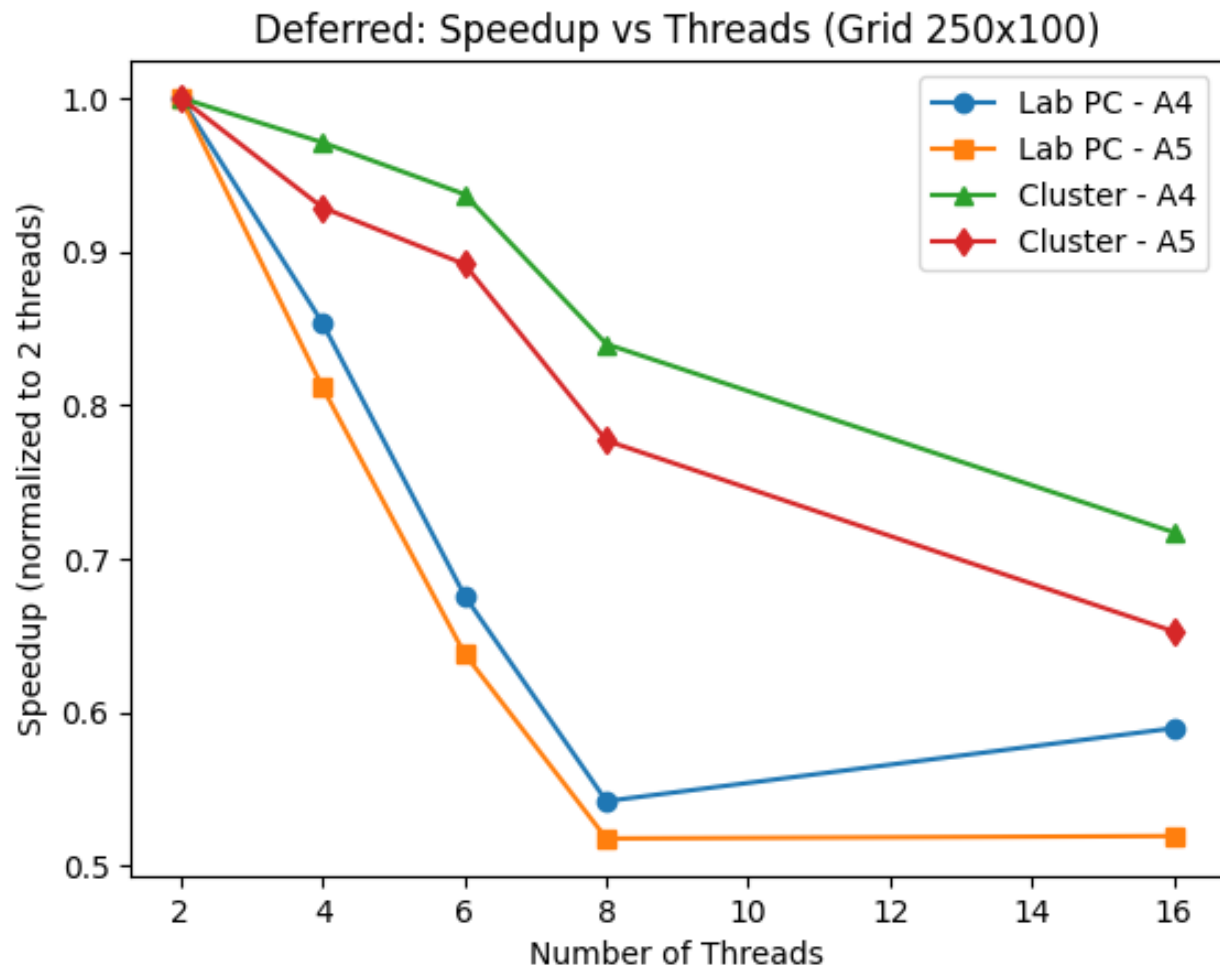


Figure 4: Deferred (250x100)

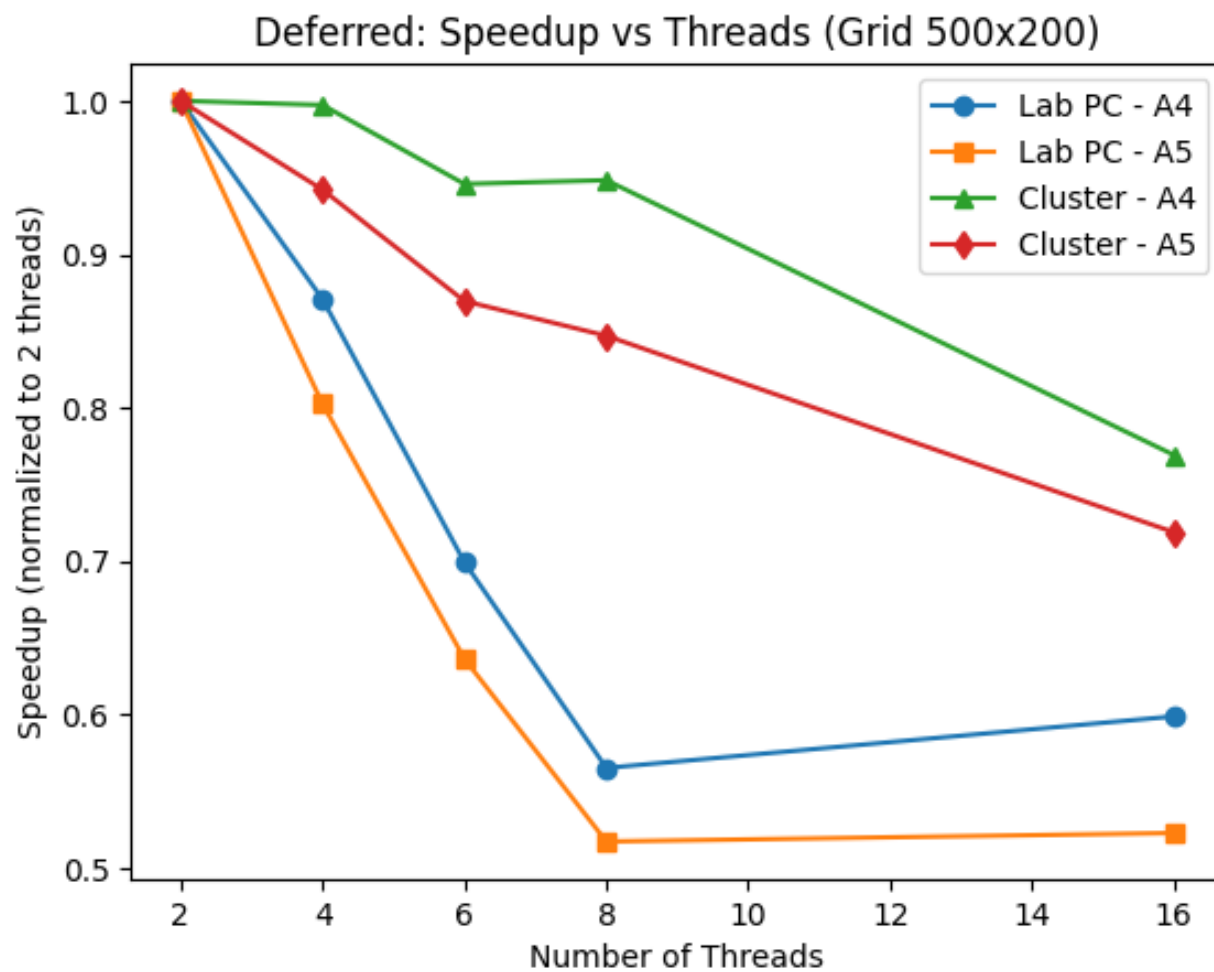


Figure 5: Deferred (500x200)

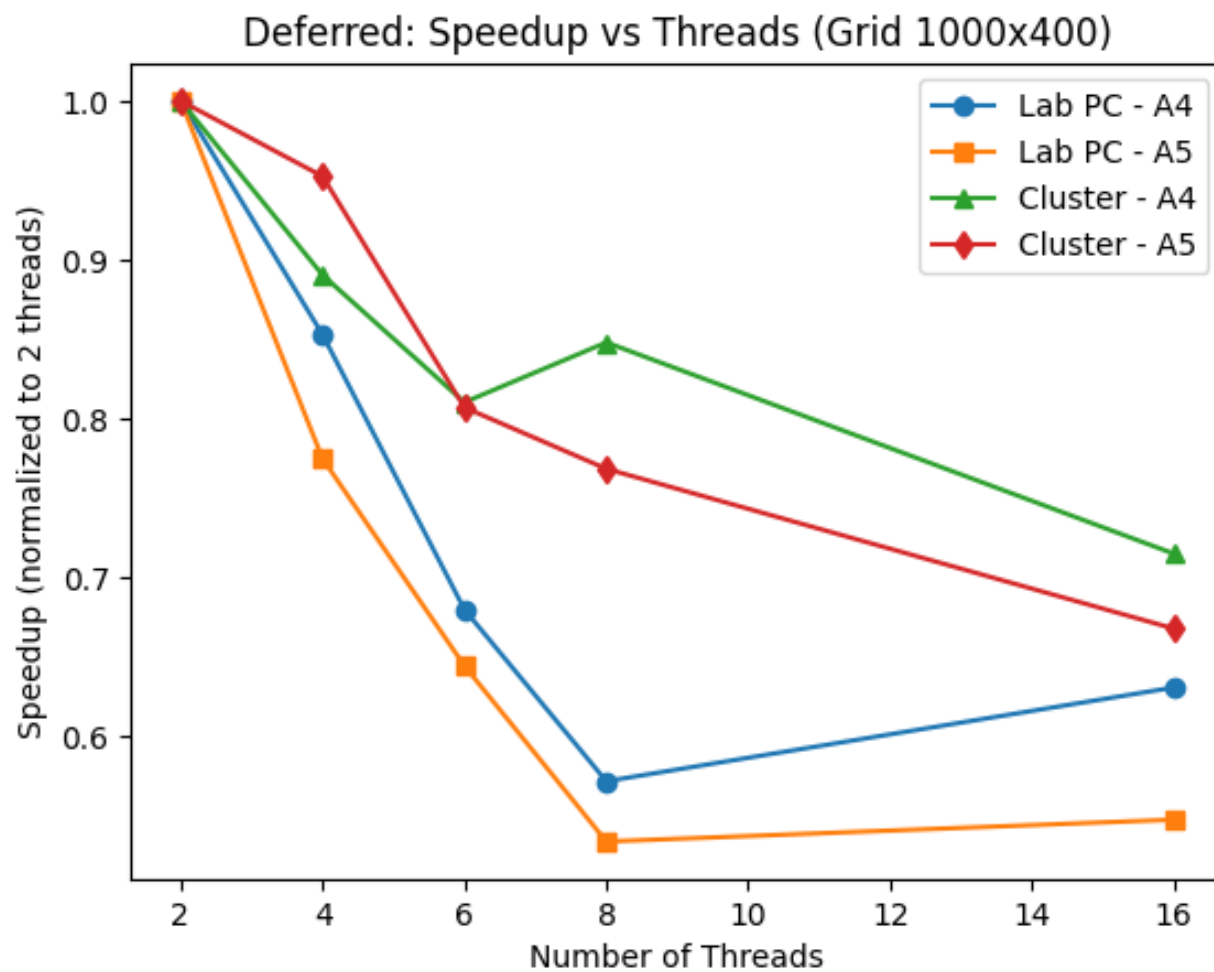


Figure 6: Deferred (1000x400)

5.3 Immediate

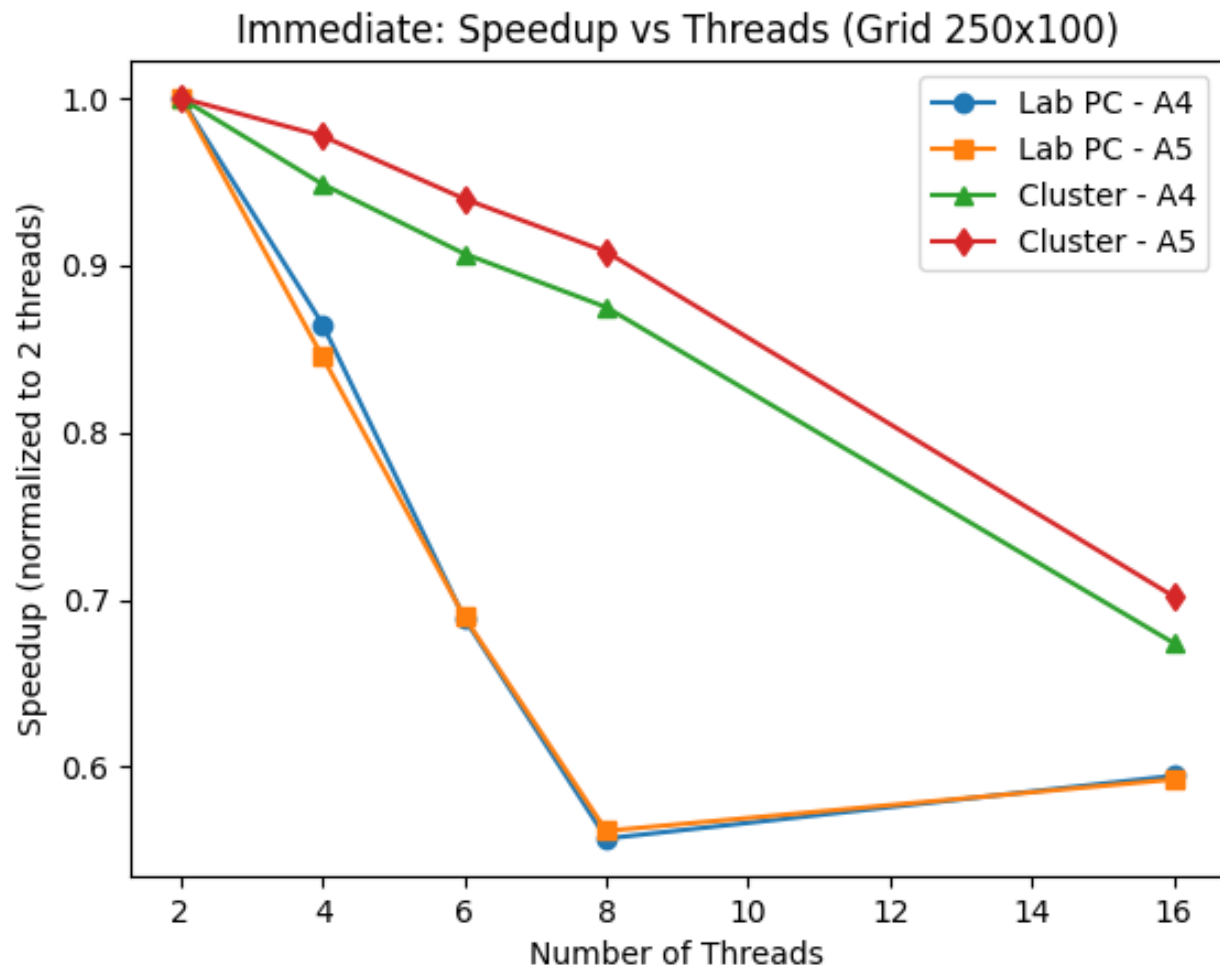


Figure 7: Immediate (250x100)

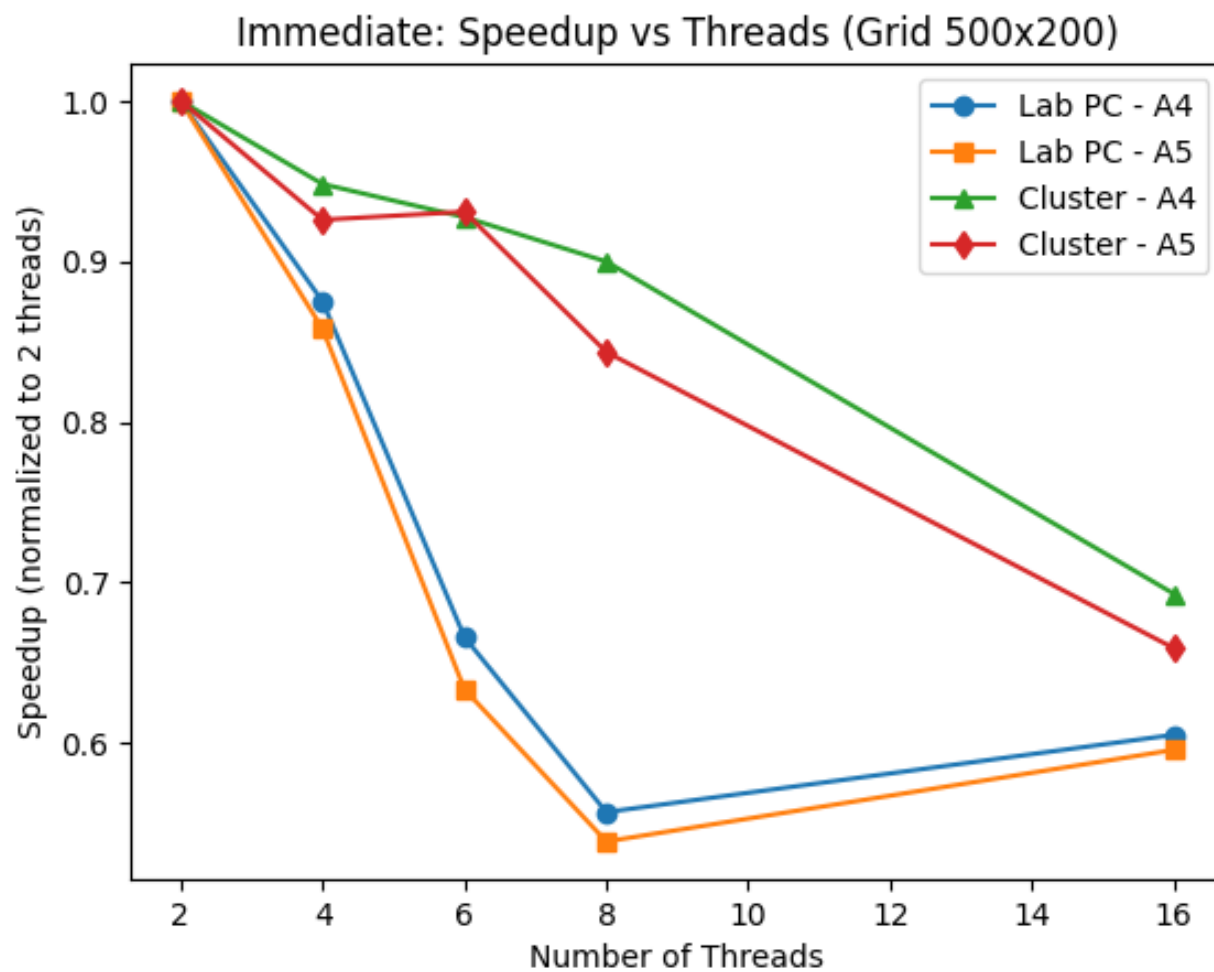


Figure 8: Immediate (500x200)

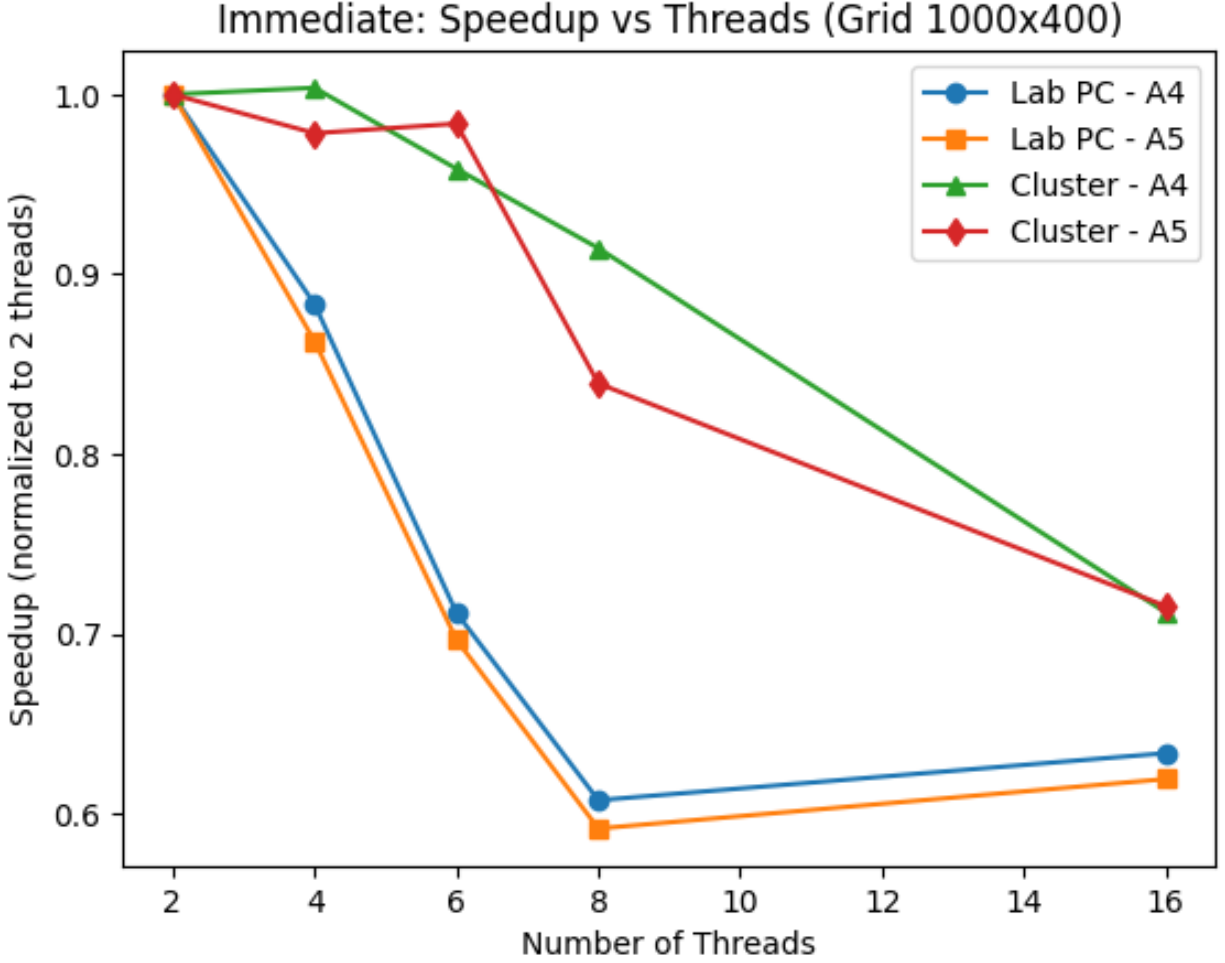


Figure 9: Immediate (1000x400)

6 Memory and Computation Analysis

6.1 Memory Requirement

Each particle stores:

- Position: $(x, y) \rightarrow 2$ doubles (16 bytes)
- Additional data (velocity/weight etc.) $\rightarrow$ assume 2 doubles (16 bytes)

Total per particle 32 bytes

Mesh grid requires:

$$N_x \times N_y \times 8 \text{ bytes}$$

6.2 Estimated FLOPs

For each particle:

- Interpolation operations
- Position updates
- Boundary checks

Approximate FLOPs per particle 20–30 operations

6.3 Analysis Table

Particles	Memory (MB)	FLOPs	Nature
10^2	≈ 0.003	$\approx 3 \times 10^3$	Compute-bound
10^4	≈ 0.3	$\approx 3 \times 10^5$	Compute-bound
10^6	≈ 32	$\approx 3 \times 10^7$	Balanced
10^8	≈ 3200	$\approx 3 \times 10^9$	Memory-bound
10^9	≈ 32000	$\approx 3 \times 10^{10}$	Strongly Memory-bound

Table 1: Memory and computation analysis

6.4 Discussion

As the number of particles increases, memory usage grows linearly, eventually dominating computation.

For small particle sizes, computation cost dominates, making the system compute-bound.

For large particle sizes, memory access becomes the bottleneck due to cache misses and limited bandwidth, making the system memory-bound.

Thus, performance shifts from compute-bound to memory-bound as problem size increases.

7 Per-Particle Performance

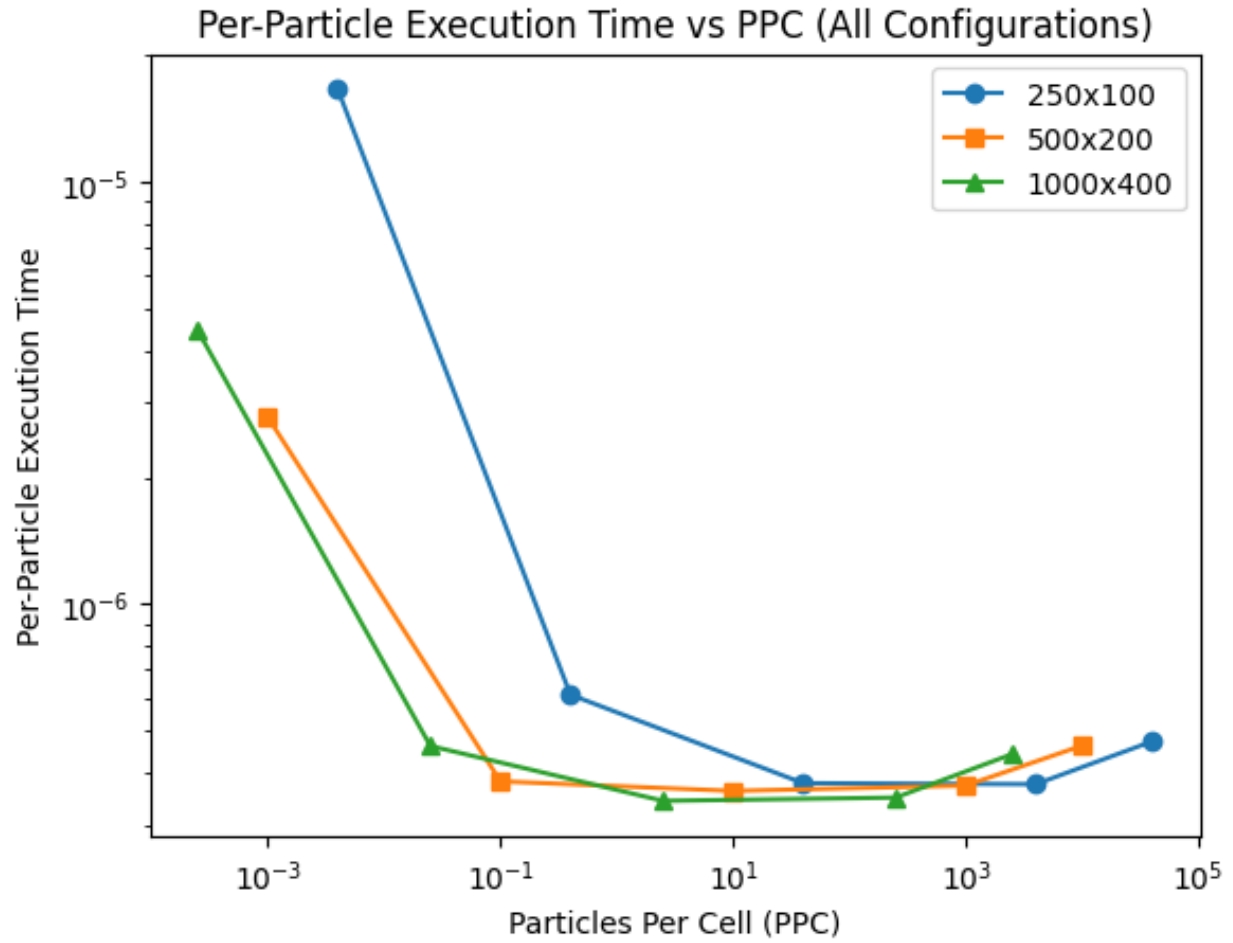


Figure 10: PPC vs Per-particle Time

Observation:

- High time at low PPC
- Minimum at moderate PPC
- Slight increase at high PPC

8 Scalability Analysis

Speedup decreases due to:

- Memory contention
- Synchronization overhead

- Cache conflicts

A5 performs worse than A4 due to insertion/deletion overhead.

9 Performance Comparison

HPC cluster performs better due to:

- Higher cores
- Better cache
- Higher bandwidth

10 Flow Diagram

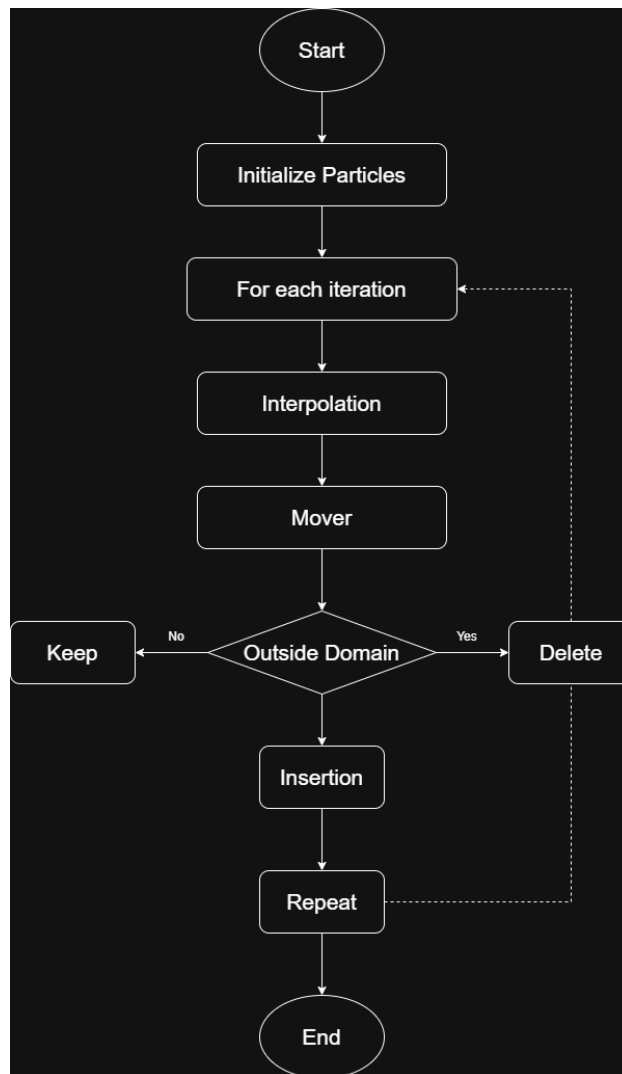


Figure 11: Mover Algorithm Flow

11 Bonus Questions

11.1 Data Structures

- Arrays: Fast but poor deletion
- Linked lists: Flexible but slow

11.2 Optimizations

- Thread-local buffers

- Spatial partitioning

12 Conclusion

Insertion and deletion reduce scalability due to memory overhead. The algorithm becomes memory-bound, and performance decreases at high thread counts.